Succinct Verification of Lattice-Based Compressed Σ-Protocols via Delegated Proofs of Correct Folding of Cryptographically Generated Public Parameters

Anders Kallesøe

Aarhus University ak@cs.au.dk

Abstract. Inner product arguments are a widely used primitive in cryptography. The bulletproofs framework and subsequently compressed Σ protocols provide a powerful folding technique that allows for succinct communication complexity of these. However, their verification complexity remains linear. The linear part of the verification is the folding computation of the CRS for the given vector commitment scheme. We explore a new avenue by which to delegate this folding to the prover via an interactive proof that incorporates the setup function of the commitment scheme in the setting where the CRS is constructed cryptographically from a small seed. We use this proof to construct a succinctly verifiable compressed Σ protocol for structured linear forms in the lattice setting.

1 Introduction

Vector commitment schemes and proofs of knowledge go hand in hand as some of the most ubiquitous cryptographic primitives with applications in many areas within cryptography. Recent years has seen inner product arguments based on linearly homomorphic vector commitments often rely on the notion of folding vectors in half around a verifier-supplied challenge. Intuitively, the idea is to create from the statement "I know $\mathbf{x}$ such that $\Psi_n(\mathbf{x}) = P$" another statement "I know $\mathbf{x}'$ such that $\Psi_{n/2}(\mathbf{x}') = Q$" where the length of $\mathbf{x}'$ is half that of $\mathbf{x}$. This second statement should imply the first one. The idea can now be applied recursively until we reach a constant length of $\mathbf{x}'^{\cdots'}$. This is a tried and by now well-studied approach to create small arguments of knowledge. However, a significant drawback is the inherent linearity that comes with evaluating the folded homomorphism $\Psi_{n/d}$. Concretely, in the context of compressed sigma protocols over Pedersen commitments the homomorphism in question is the commitment function which constitutes evaluating an MSM between the vector to be committed and the generators given in the public parameters of the scheme. Since n generators are required to commit to vectors of length n folding these requires touching n values. This quickly becomes prohibitive for large values of n. A natural question to ask is whether this folding could be verifiably delegated to the prover. However, without extra assumptions this quickly leaves us back

at square one i. e. if we use a proof system over vector commitments to prove the folding of generators then we will have to commit to vectors of length at least n. Thus, such an approach requires additional machinery. In practice, it is common to sample the public parameters for transparent vector commitment schemes cryptographically from a small seed. Indeed, if we cast this technique in the random oracle model such sampling clearly creates a sound set of public parameters. This means that while the size of the folding computation is linear, it is completely determined by a small set of values namely the small seed and the logarithmic number of challenges sent by the verifier. In this work, we explore the possibility of leveraging this observation as a new technique to create inner product arguments that are succinct both in size and verification.

1.1 Contributions

We explore the possibility of whether cryptographically generated common reference strings can be utilized to make the verification of inner product arguments based on the bulletproofs folding technique succinct. Specifically, we ask the question

Given the assumption that a cryptographically generated CRS constitutes sound public parameters for a homomorphic vector commitment scheme, is it possible to construct a succinct interactive proof of correct folding of the CRS?

In this work we demonstrate that this can be feasibly done in the lattice setting using the Ajtai commitment scheme and the Poseidon hash function, which serves to demonstrate a potential new avenue from which to consider the construction of succinct proofs of knowledge.

1.2 Technical Overview

The main technical contribution is a sumcheck-based interactive proof for the correct evaluation of the matrix-vector product

$$A\mathbf{e} \tag{1}$$

over a cyclotomic ring $\mathcal{R}_q$. We believe that the techniques developed in this endeavor can be of independent interest in the broader context of constructing succinct proof systems from lattice assumptions. The idea is to use the same observation as in [19] and utilize the fact that the multiplicative depth of matrix-vector multiplication is only 1. This alows us to consider the computation over $\mathbb{F}_q$ and postpone polynomial modular reduction to be performed outside the proof system. The approach that we take is to consider multiplications over $\mathcal{R}_q$ over the NTT form of their elements. Before modular reduction we can view these as a parallel set of convolutions and ultimately this is the computation that we construct a sumcheck argument for. Specifically, a convolution between two coefficient vectors can be written as follows:

$$(ab)(k) = \sum_{i,j} a(i)b(j)\mathbb{1}_{i+j=k} \tag{2}$$

Our approach is then to construct an indicator polynomial that extends the indicator $\mathbb{1}_{i+j=k}$ over bit string inputs i, j, k to obtain a sumcheck compatible expression of the above sum. Coupled with prior techniques due to Thaler [26] for sumcheck proofs for matrix-vector multiplication we can obtain a protocol for proving the correct evaluation of eq. (1) that reduces the problem to evaluations of the extensions $\tilde{A}, \tilde{\mathbf{e}}$ over $\mathbb{F}_q$. Here, we use the structure of $\mathbf{e}$ to design a succinct evaluation algorithm for $\tilde{\mathbf{e}}$. For $\tilde{A}$ we use the assumption that it is generated cryptographically from a small seed and invoke the GKR protocol. Furthermore, we note that the algorithms for evaluating $\tilde{A}, \tilde{e}$ can naturally be extended into extensions of $\mathbb{F}_q$ for amplified soundness. For concreteness, we show how this technique can be instantiated using the Poseidon hash function.

1.3 Related Work

Several lines of work aim to and succeed in achieving succinct verification of the linear part of the bulletproofs/CSP verifier. In the discrete log setting Dory [10] approaches the problem by designing a succinct representation of the public parameters of the commitment scheme in order to delegate folding to the prover. This technique is based on pairings but remains transparent. Closely related to our goal is [13] where the authors also aim to improve the verification complexity compressed of Σ protocols. Their setting and techniques are however quite different. They work in the SRS setting and develop a technique that allows for efficiently evaluating and folding *committed* linear forms using pairings. In [14] the authors use a FRI based argument to construct an IP for the linear part of the verifier's computation in the elliptic curve setting. To this end they apply a variant of BaseFold [27] adapted to group arithmetic. Bootle et al. [8] also design a succinct delegation protocol of the verifier's computation in the lattice setting. They achieve this via holography and a generalization of the techniques from Dory [21]. [8] achieve succinct verification via the introduction of a new assumption called Vanishing-SIS. This assumption imposes structure on the matrix used as the CRS for the Ajtai commitment scheme and this structure allows for efficient folding. Concurrent work [23] — hachi — constructs a sumcheck-based interactive argument for a very similar expression as ours, namely a matrix vector product over a cyclotomic ring, and uses this to advance the state of the art in lattice-based polynomial commitment schemes by applying it to the relation developed in Greyhound [24]. Their sumcheck argument is more efficient than ours and we conjecture that their techniques could be adapted to make this work more efficient. Another piece of concurrent work [25] — SuperNeo — also uses scalar extensions to unify a sumcheck protocol over an extension of $\mathbb{F}_q$ with proving arithmetic over a cyclotomic ring $\mathcal{R}_q$ in a similar fashion to us.

2 Preliminaries

2.1 Notation

We will often use bold variables e. g. $\mathbf{x}$ to denote vectors. For a vector $\mathbf{x}$ of even length n we take $\mathbf{x}_L$ to mean the vector of length $n/2$ containing the left half of elements of $\mathbf{x}$ and similarly for $\mathbf{x}_R$.

2.2 Interactive Proofs

Definition 1. *We call a protocol $\langle P, V \rangle$ an* interactive proof *if it maintains the following properties:*
Completeness:

$$\Pr_r \left[r \leftarrow \{0,1\}^* \ : \ \langle P(pp, w), V(pp, r) \rangle(x) = 1 \right] \geq 1 - \mathsf{negl}(\lambda)$$

Soundness: *For any adversary P^* it holds*

$$\Pr_r \left[\begin{matrix} r \leftarrow \{0,1\}^* \\ w \leftarrow \{0,1\}^* \end{matrix} \ : \ \langle P^*(pp, w), V(pp, r) \rangle(x) = 1 \right] \leq \mathsf{negl}(\lambda)$$

2.3 Interactive Proofs of Knowledge

Definition 2. *We call a protocol $\langle P, V \rangle$ an* interactive proof of knowledge *if it is complete and maintains the following property:*

Knowledge Soundness: For any adversary A there exists a *knowledge extractor* E which is PPT and such that if

$$\Pr_{w,r} \left[\begin{matrix} r \leftarrow \{0,1\}^* \\ w \leftarrow \{0,1\}^* \end{matrix} \ : \ \langle A(pp, w), V(pp, r) \rangle(x) = 1 \right] \geq \mathsf{negl}(\lambda)$$

then

$$\Pr \left[w' \leftarrow E^A(x) \ : \ (x; w') \in R \right] \geq \mathsf{negl}(\lambda)$$

2.4 Proofs vs Arguments

The notions of interactive proofs and proofs of knowledge can have their soundness properties reformulated to only hold against polynomially bounded adversaries. Such protocols are called arguments. Sometimes the terminology is conflated. In this work we use the terms interchangeably.

2.5 Module Short Integer Solution Assumption

The Module Short Integer Solution Assumption states that there exists no PPT algorithm that can solve the following problem with non-negligible probability

Definition 3 (Module Short Integer Solution). *For $\mathcal{R} = \mathbb{Z}[X]/(\Phi_d)$ the $\mathsf{MSIS}^{\infty,\kappa,q}_{m,B}$ problem is that of finding a non-zero vector $z \in \mathcal{R}$ such that $Az = 0 \mod q$ where $||z||_\infty \leq B$ and where $A \in \mathcal{R}_q^{\kappa \times m}$ is sampled uniformly at random*

2.6 Cyclotomic Polynomial Rings

Definition 4 (Cyclotomic ring). *For a prime power η we denote by Φ_η the degree d n-th cyclotomic polynomial. $\mathcal{R}_q = \mathbb{F}_q[X]/(\phi_\eta)$, the ring of polynomials $\mod \phi_\eta$ with coefficients in $\mathbb{F}_q$, is called a* cyclotomic ring.

2.7 Number Theoretic Transform

Definition 5 (Number Theoretic Transform). *For a polynomial $f \in \mathcal{R}_q$ the* Number Theoretic Transform *of f is defined as follows:*

$$\begin{Bmatrix} f_1 \\ \vdots \\ f_\tau \end{Bmatrix} \in \mathbb{F}^\tau_{q^{d/\tau}}$$

For $f_i = f \mod X^{d/\tau} - r_i$

For $f = (f_1, \ldots, f_\tau) \in \mathbb{F}_q[X]^\tau$ we define

$$\mathsf{NTTModReduce}(f) \rightarrow \begin{Bmatrix} f_1 & \mod & \mod X^{d/\tau} - r_1 \\ & & \vdots \\ f_\tau & \mod & \mod X^{d/\tau} - r_\tau \end{Bmatrix} \in \mathbb{F}^\tau_{q^{d/\tau}} \tag{3}$$

We denote by INTT the inverse map of NTT

2.8 Ajtai Commitments

Definition 6 (Ajtai Commitments). *let $\mathcal{R}_q = \mathbb{F}_q[X]/(\Phi_\eta)$ where Φ_η is the n-th cyclotomic polynomial of degree d. Let $A \in_R \mathcal{R}_q^{\kappa \times m}$. The homomorphism:*

$$\mathbf{x} \rightarrow A\mathbf{x} \tag{4}$$

is a computationally binding *commitment scheme commonly known as the Ajtai commitment scheme [2].*

2.9 Compressed Sigma Protocols

Compressed Σ protocols [4] unify the folding based techniques of bulletproofs [10] with Σ protocol theory. They works by replacing the last message of a standard Σ protocol with a PoK of the last message that has only half the size of the message itself. The final message of this PoK has the same form as that of the original Σ protocol so the process can be repeated recursively effectively halving the size of the proof each time until a constant size is reached. This can be used to construct logarithmically sized opening proofs for any one-way homomorphism — typically vector commitment schemes.

Folding Compressed Σ protocols work via "folding". Essentially, one can think of the process as folding the linear homomorphism Ψ_n that we want to prove knowledge of a pre-image of in half at every step with the lower half being multiplied by a verifier-supplied challenge e resulting in a homomorphism of half size. Folding all the way down results in a homomorphism over a single input element. Computing this folding takes linear time and is the root of the linear verification complexity of compressed Σ protocols. The computation is very structured and the result can be interpreted as computing $\Psi_n(\mathbf{e})$ where $\mathbf{e}$ is a vector that contains all the multiplicative combinations of the $\log n$ challenges sent by the verifier. Specifically, for Ajtai commitments this corresponds to computing $A\mathbf{e}$.

2.10 Multilinear Polynomial Extensions

Definition 7 (Multilinear polynomial). *A multilinear polynomial $f(x_1, \ldots, x_l)$ is a multivariate polynomial whose degree in any of its variables is at most 1*

A multilinear polynomial is uniquely defined by its evaluations over the boolean hypercube. In fact, any assignment of entries of a vector over $\mathbb{F}_q^n$ where $n = 2^l$ uniquely defines a multilinear polynomial in l variables over $\mathbb{F}_q$. This gives rise to the notion of *multilinear extensions*. Any function $F : \{0,1\}^\ell \to \mathbb{F}_q$ has a multilinear extension g, which is a multilinear polynomial such that $g(x_1, \ldots, x_l) = F(x_1, \ldots, x_l)$ for all $x_1, \ldots, x_l \in \{0,1\}$. It can be concretely interpolated as

$$g(x_1, \ldots, x_l) = \sum_{y \in \{0,1\}^\ell} F(y)\tilde{eq}(x, y) \tag{5}$$

Where

$$\tilde{eq}(x, y) = \prod_{i=1}^{l}(x_i y_i + (1 - x_i)(1 - y_i)) \tag{6}$$

$\tilde{eq}$ has the property that over boolean inputs that $\tilde{eq}(x, y) = 1$ if $x = y$ as bit strings and 0 otherwise.

2.11 The Sumcheck Protocol

The sumcheck protocol [22] is a seminal piece of work that allows a possibly malicious prover to succinctly prove the result S of summing all the evaluations of a given polynomial g over a field $\mathbb{F}$ in say n variables on all points in a given set of points H^n where $H \subset \mathbb{F}$. H is very often taken to be $\{0, 1\}$. At a high level it works by having the prover sum out every variable except the first. This results in a univariate polynomial g_1, which the prover will send. The verifier will now check that $g_1(0) + g_1(1) = S$ (in the case where $H = \{0, 1\}$). Next, the verifier should check that g_1 is well formed. This is done by having the verifier pick a random challenge in $\mathbb{F}$ which defines a new polynomial in $n - 1$ variables which can be recursively checked for consistency using sumcheck. After having gone through all the variables of the polynomial the verifier needs to perform a single evaluation of g in the point defined by all the challenges sent by the verifier. This can be done either directly by V or by oracle access to g.

3 Interactive Proof with Succinct Communication and Verification for the Evaluation of $\tilde{A}$

3.1 Secure Generation of Public Parameters in the Random Oracle Model

The Ajtai commitment scheme lives in the CRS model meaning that we rely on the assumption that the prover and the verifier alike have access to a public, random string the size of which is proportional to the length of the vectors to which we wish to able to make commitments. This public string makes up the public parameters of the commitment scheme. For Ajtai commitments specifically we interpret it as the matrix $A \in \mathcal{R}_q^{\kappa \times m}$ that make up the public parameters. Thus, in order for the commitment scheme to be used this matrix must come into existence. The only assumption made on A is that it is sampled independently and uniformly at random from $\mathcal{R}_q^{\kappa \times m}$. Thus, it qualifies as being *transparent* in the sense that there is no secret *trapdoor*. Specifically, there's no structural information about A that should be kept secret as is the case for e. g. KZG commitments. This enables a very simple sampling procedure in the random oracle model that goes as follows:

Definition 8 (ROM sampling).

$$\mathsf{sample} : \mathbb{F}_{q^k} \to \mathcal{R}_q^{\kappa \times m} \tag{7}$$

$$\mathsf{sample}(s) \to \begin{bmatrix} \mathsf{RO}(s) & \mathsf{RO}(s^2) & \dots & \mathsf{RO}(s^m) \\ \vdots & \vdots & \vdots & \vdots \\ \mathsf{RO}(s^{m(\kappa-1)+1}) & \mathsf{RO}(s^{m(\kappa-1)+2}) & \dots & \mathsf{RO}(s^{\kappa m}) \end{bmatrix} \tag{8}$$

Clearly, if s is a generator of the multiplicative group over $\mathbb{F}_{q^k}$ and $|\mathbb{F}_{q^k}| > \kappa m$ then $\mathsf{sample}(s)$ outputs a uniformly random matrix in $\mathcal{R}_q^{\kappa \times m}$. Thus, sample is

a sound function with which to generate public parameters for the Ajtai commitment scheme. Furthermore, it doesn't matter if we consider the output to be in coefficient form or NTT form. Namely, since there exists a bijection between the two then sampling uniformly at random from one and then mapping to the other produces a uniformly random representation of either.

3.2 Secure Generation of Public Parameters in the Plain Model

To make our protocol work we need to instantiate a sampling procedure for public parameters in the plain model.

Definition 9 (Plain Model Sampling). *Let $\mathcal{H}$ be a family of correlation intractable hash functions from $\mathbb{F}_{q^k} \to \mathcal{R}_p$. For a particular $h \in \mathcal{H}$ we define the function* sample_h *as follows:*

$$h : \mathbb{F}_{q^k} \to cR_q \tag{9}$$

$$\mathsf{sample}_h : \mathbb{F}_{q^k} \to \mathcal{R}_q^{\kappa \times m} \tag{10}$$

$$\mathsf{sample}_h(s) \to \begin{bmatrix} h(s) & h(s^2) & \dots & h(s^m) \\ \vdots & \vdots & \vdots & \vdots \\ h(s^{m(\kappa-1)+1}) & h(s^{m(\kappa-1)+2}) & \dots & h(s^{m\kappa}) \end{bmatrix} \tag{11}$$

We heuristically conjecture that the function sample_h samples sound public parameters for the Ajtai commitment scheme when evaluated on a generator of $\mathbb{F}^*_{q^k}$ when h is instantiated from $\mathcal{H}$. We view it as an interesting open question to determine the minimum requirements on the function h that ensures that sample_h produces a sound CRS.

3.3 Standard Form vs NTT Form

For efficiency reasons we prefer to maintain the matrix of ring elements in NTT form rather than standard form. Since there exists a bijection between elements of $\mathcal{R}_q$ in NTT form and elements in coefficient form it follows that sampling a uniformly random element of one implicitly means sampling a uniformly random element from the other. Thus, throughout we will think of the sampling process as sampling individual field elements for the NTT representation. That is, we consider the following sampler

$$h : \mathbb{F}_{q^k} \to \mathbb{F}_{q^{d/\tau}} \tag{12}$$

$$\mathsf{sample}_h : \mathbb{F}_{q^k} \to (\mathbb{F}^{\tau}_{q^{d/\tau}})^{\kappa \times m} \tag{13}$$

$$\mathsf{sample}_h(s) \to \begin{bmatrix} h(s) & h(s^2) & \dots & h(s^{\tau m}) \\ \vdots & \vdots & \vdots & \vdots \\ h(s^{\tau m(\kappa-1)+1}) & h(s^{\tau m(\kappa-1)+1}) & \dots & h(s^{\tau m\kappa}) \end{bmatrix} \tag{14}$$

Here, the output of h is such that it matches one element of $\mathbb{F}_{q^{d/\tau}}$. We note that one could have h matching any multiple or fraction of such.

3.4 The Folding Computation of Compressed Sigma Protocols

Throughout the execution of a compressed Σ protocol the verifier will send challenges $e_1, \ldots, e_\ell$. At each step, the current matrix A_i will be folded as $A_{i+1} = e_i(A_i)_L + (A_i)_R$. Ultimately, the result of this folding is the matrix vector product

$$A\mathbf{e} \tag{15}$$

Where $\mathbf{e} = \mathsf{fold}(e_1, \ldots, e_\ell)$. Here, we define fold as follows:

$$\mathsf{fold}(x_1, \ldots, x_l) = (x_1 \ldots x_l, x_1 \ldots x_{\ell-1}, x_1 \ldots x_{\ell-2}x_l, \ldots, 1) \tag{16}$$

We also say that $\mathsf{fold}(e_1, \ldots, e_\ell)$ is *tensor-structured* since it can be written as the tensor $\begin{bmatrix} e_1 \\ 1 \end{bmatrix} \otimes \cdots \otimes \begin{bmatrix} e_\ell \\ 1 \end{bmatrix}$

3.5 Succinct Verification of Folding of Ajtai Public Parameters

We consider a circuit that takes two input vectors with elements in $\mathbb{F}_q$. One S is the concatenation of the extension field elements that are used as input to the cryptographic hash function from which we generate the public parameters for our Ajtai Commitments. The second C is the concatenation of all the multiplicative combinations over $\mathcal{R}_q$ of $e_1, ..., e_\ell$ the $\ell = \log m$ challenges involved in folding during the execution of the compressed Σ protocol that we wish to verify succinctly. We will construct a layered circuit that takes these inputs, evaluates the chosen hash function in parallel over the input vector extended by $\tilde{S}$ and then performs the matrix vector product over $\mathcal{R}_q$ between the computed matrix and the vector of ring elements extended over $\mathbb{F}_q$ by C. Since each multiplication involved in this matrix vector product is only ever pairwise we can optimize the circuit by taking the ring elements as input in NTT form. Thus, we don't have to perform the transformation to and from NTT inside the circuit but the verifier can simply perform the inverse NTT outside the circuit. We will consider the following 3 polynomials:

$$\tilde{A}(\mathsf{row}, \mathsf{col}, \mathsf{dep}) \tag{17}$$

$$\tilde{C}(\mathsf{col}, \mathsf{dep}) \tag{18}$$

$$\tilde{S}(\mathsf{col}, \hat{\mathsf{dep}}) \tag{19}$$

Here, we can think of $\mathsf{row} \in \{0,1\}^{\log \kappa}$ as the bits determining which row of A we are in. $\mathsf{col} \in \{0,1\}^{\ell}$ will determine the column of A and correspondingly the entries of C. $\mathsf{dep} \in \{0,1\}^{\log d}$ determines the individual $\mathbb{F}_q$ element in the NTT form of the given ring element. If $\mathcal{R}_q$ doesn't fully split then we need a further chunk of bits to properly index into individual $\mathbb{F}_q$ elements making up the NTT form of a ring element. We will explain as we get there. Similarly, $\hat{\mathsf{dep}}$ indexes into individual $\mathbb{F}_q$ elements of an extension field $\mathbb{F}_{q^\kappa}$ element in S.

Fully Splitting Ring If $\mathcal{R}_q$ is fully splitting, then we can express the output of the folding process of A around $e_1, \ldots, e_\ell$ as follows:

$$O(\mathsf{row}, \mathsf{dep}) = \sum_{\mathsf{col} \in \{0,1\}^\ell} A(\mathsf{row}, \mathsf{col}, \mathsf{dep}) C(\mathsf{col}, \mathsf{dep}) \tag{20}$$

since the multiplication between two ring elements in NTT form amounts to the Schur product between the two evaluation vectors over $\mathbb{F}_q$. We can consider the same expression in terms of the multilinear extensions of these vectors and matrices interpolated multilinearly:

$$\tilde{O}(\mathsf{row}, \mathsf{dep}) = \sum_{\mathsf{col} \in \{0,1\}^\ell, \mathsf{dep}' \in \{0,1\}^{\log d}} \tilde{A}(\mathsf{row}, \mathsf{col}, \mathsf{dep}') \tilde{C}(\mathsf{col}, \mathsf{dep}') \tilde{\mathsf{eq}}(\mathsf{dep}', \mathsf{dep}) \tag{21}$$

This polynomial extends the folded public parameters. Furthermore, the right-hand side is multilinear in $\mathsf{row}, \mathsf{dep}$. Thus, $\tilde{O}$ as defined above is the unique multilinear extension of the folded public parameters.

Partially Splitting Ring If $\mathcal{R}_q$ is only partially splitting which will often be the case in instantiations of the MSIS problem then multiplication of ring elements can no longer be broken down into pairwise multiplications over $\mathbb{F}_q$. Instead, it is broken down into pairwise multiplications over some extension field of $\mathbb{F}_q$. Let's call this field $\mathbb{F}_{q^{d/\tau}}$. We wish to similarly utilize the SIMD structure of the folding computation now over multiplications of $\mathbb{F}_{q^{d/\tau}}$ elements. We do that by constructing an efficient interactive proof for linear convolutions. Multiplying two elements $a, b \in \mathbb{F}_{q^{d/\tau}}$ can be expressed as the convolution between their coefficient vectors before modular reduction

$$(ab)[k] = \sum_{i=0}^{k} a[i] b[k-i] \tag{22}$$

Where $a[i]$ denotes the i'th coefficient of a. This can equivalently be expressed as:

$$(ab)[k] = \sum_{i,j \in [0; \log d/\tau] : i+j=k} a[i] b[j] \tag{23}$$

This is the expression that we will utilize to express the convolution between x and y as a sum compatible with the sumcheck protocol. Using an indicator variable we can write:

$$\tilde{(ab)}(k) = \sum_{i,j \in \{0,1\}^{\log d/\tau}} \tilde{a}(i) \tilde{b}(j) \mathbb{1}_{\mathsf{int}(i)+\mathsf{int}(j)=\mathsf{int}(k)} \tag{24}$$

Thus, the only thing we really need is to define the multilinear extension of the indicator in the above expression. We are given boolean inputs and have to determine whether the sum of the integers represented by the first two blocks

of bits equal that of the third block. So we need to express addition with carry in a way where the expression stays multilinear. Taking x, y, z to be the bit strings in question, we will express constraints that are satisfied if and only if $\mathsf{int}(x) + \mathsf{int}(y) = \mathsf{int}(z)$. To make this easier, we will include the carries in this condition. For an assignment of bits x_i, y_i, z_i, c_i we can determine if they correctly encode an addition with carry. If that's the case we want our polynomial to output 1. If not, it should output 0. The constraints that enforce this property can be expressed as follows:

$$x_i + y_i + c_i = z_i + 2c_{i+1} \forall i \in [0; \log(d/\tau)) \tag{25}$$

and

$$c_0 = 0 \wedge c_{\log(d/\tau)} = z_{\log(d/\tau)} \tag{26}$$

We can verify that this enforces correct addition by a case analysis: If all of x_i, y_i, c_i are 1 the constraint is only satisfied if z_i, c_{i+1} are 1, corresponding to the correct assignments in a carry addition. If only two of x_i, y_i, c_i are 1 the constraint is only satisfied if only c_{i+1} is 1 again corresponding to the correct assignments in a carry addition. If only one of x_i, y_i, c_i equals 1 then the assignment is only satisfied if only z_i is 1 again corresponding to the correct assignments in a carry addition. Finally, if none of x_i, y_i, c_i are 1 then the constraint is only satisfied if none of z_i, c_{i+1} are 1, again corresponding to the correct assignments in a carry addition. We can express the constraint via the following multilinear polynomial in z:

$$\mathsf{CarrySum}_{x,y}(z) = \sum_{c \in \{0,1\}^{\log(d/\tau)+1}} \left[\prod_{i=0}^{\log(d/\tau)} \tilde{\mathsf{h}}(z_i, x_i, y_i, c_i, c_{i+1}) \right] \tilde{\mathsf{eq}}(c_0, 0) \tilde{\mathsf{eq}}(z_{\log(d/\tau)+1}, c_{\log(d/\tau)+1}) \tag{27}$$

Where $\tilde{h}$ is a polynomial linear in it's first variable that outputs 1 if the constraint is satisfied between it's 5 input bits and 0 otherwise. Such a polynomial is guaranteed to exist due to Lagrange interpolation. Now, we can express the convolution as

$$\tilde{(ab)}(k) = \sum_{i,j \in \{0,1\}^{k^*}} \tilde{a}(i)\tilde{b}(j)\mathsf{CarrySum}_{i,j}(k) \tag{28}$$

We need this in a SIMD style proof i. e. we are considering a circuit where the convolution computation is run in parallel a number times across the matrix vector product between A and C. We will consider the matrix A to be indexed by four bit strings $\mathsf{row}, \mathsf{col}, \mathsf{dep}, f$ where row determines the row of A, col determines the column of A. For an entry $a \in \mathbb{F}^{\tau}_{q^{d/\tau}}$ in A, dep indexes into the individual $\mathbb{F}_{q^{d/\tau}}$ elements making up a. Finally, f indexes into the individual $\mathbb{F}_q$ elements making up such an extension field element. We can express the multilinear extension of the output of the Matrix vector product between A and C over

$\mathcal{R}_q$ in NTT form before modular reduction after convolutions as the following polynomial:

$$\tilde{O}(\mathsf{dep}, f) = \sum_{\mathsf{row}\in\{0,1\}^{\log\kappa},\mathsf{col}\in\{0,1\}^{\ell},\mathsf{dep}'\in\{0,1\}^{\log(t)},f'\in\{0,1\}^{\log(d/\tau)},f''\in\{0,1\}^{\log(d/\tau)}} \tilde{A}(\mathsf{row},\mathsf{col},\mathsf{dep}',f')\tilde{C}(\mathsf{col},\mathsf{dep}',f'')\mathsf{CarrySum}_{f',f''}(f)\tilde{\mathsf{eq}}(\mathsf{dep}',\mathsf{dep}) \quad (29)$$

Now we have an expression that's compatible with the sumcheck protocol where $\mathsf{CarrySum}_{f',f''}$, $\tilde{\mathsf{eq}}$, $\tilde{C}$ can be evaluated directly by the verifier whereas the evaluation of $\tilde{A}$ can be delegated to an invocation of GKR on a circuit that evaluates the chosen hash function that we've used to generate A. $\tilde{O}$ is an extension of the field elements making up the convoluted entries of the NTT form of the claimed folded public parameters before modular reduction. Thus, after verification of the IP the verifier can reduce and compute the inverse NTT to obtain the final result. Notably, this method doesn't enforce any method of computing the convolutions for the prover.

3.6 Evaluating $\tilde{C}$ Efficiently over $\mathbb{F}_q$

When running our GKR style interactive proof, the verifier needs to be able to evaluate $\tilde{O}, \tilde{A}, \tilde{C}$ in a random point. $\tilde{O}$ is proportional to the size of the folded output and can be evaluated directly by the verifier. The evaluation of $\tilde{A}$ will be deferred to the previous layer of the computation. We now describe how to evaluate $\tilde{C}$ over $\mathbb{F}_q$. Recall that C is the concatenation of all the multiplicative combinations of $e_1, \ldots, e_\ell$ over $\mathcal{R}_q$ in NTT form. However, to begin with let's first assume that the entries are in standard form. We want to evaluate the multilinear extension of this vector over $\mathbb{F}_q$. Here, we assume that d is a power of 2. For a point

$$(r_1, \ldots, r_{\log d+\ell}) \in \mathbb{F}_q^{\log d+\ell} \quad (30)$$

we wish to compute the inner product between C and

$$(1-r_1)\ldots(1-r_{\log d+\ell}), \quad r_1\ldots(1-r_{\log d+\ell}), \quad \ldots, \quad r_1\ldots r_{\log d+\ell} \quad (31)$$

i. e. all the multiplicative combinations between $(1-r_1, \ldots, 1-r_{\log d+\ell})$ and $(r_1, \ldots, r_{\log d+\ell})$ where each entry in the product is taken either from $(1-r_1, \ldots, 1-r_{\log d+\ell})$ or $(r_1, \ldots, r_{\log d+\ell})$. We now utilize the fact that $\mathcal{R}_q$ is an $\mathbb{F}_q$-module. If we consider the multiplicative combinations as sketched above over only $r_{\log(d)+1}, \ldots, r_{\ell+\log(d)}$ then we can compute the linear combination between this resulting vector and C over $\mathcal{R}_q$ via the following simple algorithm:

1. $v_0 := 1$
2. For $i \in [\log(d)+1, \ldots, \log(d)+\ell]$
   (a) $v_i := v_{i-1}(1-r_i)e_i + v_{i-1}r_i$

3. Output $v_{\log(d)+\ell}$

We now observe that since d is a power of 2 then $v_{\log(d)+\ell}$ actually corresponds to a partial evaluation of the multilinear polynomial over $\mathbb{F}_q$ that extends C. All that remains to evaluate this polynomial is a similar linear combination of the entries in $v_{\log(d)+\ell}$. Thus, the verifier can evaluate $\tilde{C}$ over $\mathbb{F}_q$ efficiently. However, we don't want to compute the NTT inside the GKR circuit. Thus, we want to do the evaluation of $\tilde{C}$ where C is taken to be the concatenation of the NTT form of the multiplicative combinations of $e_1, \ldots, e_\ell$. Thankfully, this requires no change at all since the NTT is an isomorphism. In fact, we can compute v_ℓ as we just described, compute the $v'_{\log(d)+\ell} = \mathsf{NTT}(v_{\log(d)+\ell})$ and then compute the proper linear combination of the entries of $v'_{\log(d)+\ell}$ to get the final evaluation. Indeed, we would probably already have done the computation over the NTT form for efficiency anyways.

3.7 Evaluating $\tilde{S}$ Efficiently over $\mathbb{F}_q$

Similarly to $\tilde{C}$, the verifier must be able to efficiently evaluate $\tilde{S}$ in order to verify the input layer in the execution of GKR. This will follow a very similar procedure to the evaluation of $\tilde{C}$. Recall that S is the concatenation of the powers of a randomly sampled generator s of the extension field $\mathbb{F}_{q^k}$. We want to evaluate $\tilde{S}$ over $\mathbb{F}_q$. We will follow the same blueprint as for $\tilde{C}$. Since $\mathbb{F}_{q^k}$ is an $\mathbb{F}_q$-module the verifier can evaluate the linear combination between the multiplicative combinations of $r_1, ..., r_{\ell \log \kappa}$ and S over $\mathbb{F}_{q^k}$ by the following procedure

1. $v_0 := 1$
2. For $i \in [l \log \kappa]$
   (a) $v_i := v_{i-1}(1 - r_i) + v_{i-1} s^{2^i} r_i$
3. Output $v_{\ell \log \kappa}$

We can consider the computed sum as an element of $\mathbb{F}_{q^k}$. If we assume that k is a power of 2 the full evaluation of $\tilde{S}$ over $\mathbb{F}_q$ is a linear combination of the entries in $v_{\ell \log \kappa}$.

3.8 GKR Style Verification of Parallel evaluations of the Poseidon hash function

For concreteness we employ the GKR protocol to prove parallel evaluations of the Poseidon hash function with the purpose of proving an evaluation of $\tilde{A}$. This is fairly straight-forward via prior techniques. We refer to Appendix C for details.

3.9 When d, τ, κ are not powers of 2

The sumcheck protocol over multilinear polynomials inherently works over powers of 2 since multilinear polynomials are defined over their evaluations on boolean hypercubes. If our parameters don't fit nicely into this frame then we need to make some adaptations. We discuss these in the following.

When $\tau, d/\tau$ are not powers of 2 τ determines the number of extension field elements that make up the NTT form of an element of $\mathcal{R}_q$ and d/τ determines the number of elements of $\mathbb{F}_q$ that make up an element of the extension field over which elements of the NTT form is defined. For illustration, let's consider a case where they are both 3. The smallest power of 2 bigger than 3 is 4. Thus, in the circuit that we are constructing a prover for, we can think of an element of the form

$$[a, b, c, 0], [d, e, f, 0], [g, h, i, 0], [0, 0, 0, 0] \tag{32}$$

as that which occupies a single entry of the matrix A. That is, we are considering each element of A as made up of 4 buckets of 4 elements of $\mathbb{F}_q$. Performing a pair-wise (over buckets) convolution between two elements of such form will yield the same result as the same computation over three buckets with the same three elements as the non-zero entries of those of the form above — albeit with some extra 0's. Thus, we could perform convolutions and linear combinations of the results on elements of the above form inside our prover circuit and then do the appropriate conversion to $\mathcal{R}_q$ elements outside the circuit.

Evaluating $\tilde{C}$ and $\tilde{S}$ In our algorithms for efficiently evaluating $\tilde{C}$ and $\tilde{S}$ we run them almost in the same way as before. The difference is the linear combination of the $\mathbb{F}_q$ elements in the computed $v_{\log(d)+\ell}$ and $v_{\ell+\log\kappa}$. In this new setting we need to 0-pad these in line with the nearest powers of 2 as described in the previous section before doing the final linear combination. This ensures that the computation corresponds to the concatenation of the correctly padded elements.

3.10 Evaluation over Extension Field

The security of the GKR protocol hinges on the Schwartz-Zippel lemma [28], which states that two different polynomials over $\mathbb{F}_q$ of total degree at most d agree in a random point with probability $\frac{d}{|\mathbb{F}_q|}$. This requires $|\mathbb{F}_q|$ to be so large that this probability is considered negligible. It's not always the case for instantiations of the M-SIS assumption that $|\mathbb{F}_q|$ is large enough to provide adequate soundness. This is typically mediated by two different approaches: Running the protocol several times in parallel or running the protocol over an extension field $\mathbb{F}_{q^n}$. Since parallel repetition incurs a high overhead when a protocol is rendered non-interactive using the Fiat-Shamir heuristic it would be beneficial to be able to run the protocol over an extension field. Thus, we need to make sure that the algorithms for evaluating $\tilde{C}$ and $\tilde{S}$ efficiently extend easily into $\mathbb{F}_{q^n}$. This can be achieved via scalar extensions. Namely, we run the algorithm over $\mathbb{F}_{q^n} \otimes_{\mathbb{F}_q} \mathbb{F}_{q^{d/\tau}} \cong \mathbb{F}_{q^n}[X]/f(X)$ (where $\mathbb{F}_{q^{d/\tau}} = \mathbb{F}_q[X]/f(x)$) for each of the NTT limbs in parallel and combine the coefficients of the resulting elements appropriately to obtain the evaluation of $\tilde{C}$ over $\mathbb{F}_{q^n}$. Indeed, $\mathbb{F}_{q^n}[X]/f(X)$ is a commutative ring and $\mathbb{F}_{q^{d/\tau}}$ elements can be embedded into $\mathbb{F}_{q^n}[X]/f(X)$ via the canonical inclusion. So for $x_i, y_i \in \mathbb{F}_{q^{d/\tau}}$ and $a, b \in \mathbb{F}_{q^n}$ we

have $[1, x_i, y_i, x_i y_i] \cdot [1, a, b, ab] = (1 + ax_i)(1 + by_i)$. If extending the coefficients of $\mathcal{R}_q$ causes further splitting then the isomorphism between the NTT representation of C and its coefficient representation breaks after running the algorithm. But we only need to evaluate the multilinear extension of the concatenation of all the elements of the NTT representation of C for the sumcheck protocol to work. Thus, the evaluation algorithm now necessitates that we perform the NTT conversion before evaluating.

3.11 Putting Things Together: Succinctly Verifiable Proof of Correct Folding

We now describe the full interactive proof for the correct folding of public parameters. We wish to construct an efficient prover for the relation

$$R_{\mathsf{fold}} = \left\{ (A, \mathbf{y}, s : \mathbf{x}) \in (\mathcal{R}_q^{\kappa \times m}, \mathcal{R}_q^{\kappa}, \mathbb{F}_{q^k} : \mathcal{R}_q^m) \;\middle|\; \begin{array}{r} A\mathbf{x} = \mathbf{y} \\ A \leftarrow \mathsf{sample}_{\mathsf{poseidon}}(s) \\ \mathsf{x} \leftarrow \mathsf{fold}(e_1, \ldots, e_\ell) \end{array} \right\} \tag{33}$$

The protocol is given in fig. 1

Parameters: $(A, \mathbf{y}, s : \mathbf{x}) \in (\mathcal{R}_q^{\kappa \times m}, \mathcal{R}_q^{\kappa}, \mathbb{F}_{q^k} : \mathcal{R}_q^m)$
Claim: $(A, \mathbf{y}, s : \mathbf{x}) \in R_{\mathsf{fold}}$
Protocol:

1. P: Send O to V
2. V: If $INTT(\mathsf{NTTModReduce}(O)) \neq \mathbf{y}$ output $\perp$
3. PV: Define the polynomial
$$h(\mathsf{dep}, f) = \sum_{\mathsf{row} \in \{0,1\}^{\log \kappa}, \mathsf{col} \in \{0,1\}^{\ell}, \mathsf{dep}' \in \{0,1\}^{\log \tau}, f' \in \{0,1\}^{\log(d/\tau)}, f'' \in \{0,1\}^{\log(d/\tau)}} \tilde{A}(\mathsf{row}, \mathsf{col}, \mathsf{dep}', f') \tilde{C}(\mathsf{col}, \mathsf{dep}', f'') \mathsf{CarrySum}_{f', f''}(f) \tilde{\mathsf{eq}}(\mathsf{dep}, \mathsf{dep}')$$
4. V: Send random challenge r_{dep}, r_f to P
5. PV: Run sumcheck to evaluate $h(r_{\mathsf{dep}}, r_f)$. Call V's challenges $(r_{\mathsf{row}}, r_{\mathsf{col}}, r_{\mathsf{dep}'}, r_{f'}, r_{f''})$
6. V: Verify that $\tilde{O}(r_{\mathsf{dep}}, r_f) = h(r_{\mathsf{dep}}, r_f)$ otherwise output $\perp$
7. V: Evaluate $\tilde{C}(r_{\mathsf{col}}, r_{\mathsf{dep}'}, r_{f''})$ using the algorithm from section 3.6
8. V: Evaluate $\mathsf{CarrySum}_{r_{f'}, r_{f''}}(r_f)$
9. V: Evaluate $\tilde{eq}(r_{\mathsf{dep}}, r_{\mathsf{dep}'})$
10. P: Send $y = \tilde{A}(r_{\mathsf{row}}, r_{\mathsf{col}}, r_{\mathsf{dep}'}, r_{f'}, r_{f''})$ to V
11. PV: Run protocol $\Pi_{\mathsf{ParallelPoseidon}}$ fig. 4 on input $(A, s, (r_{\mathsf{row}}, r_{\mathsf{col}}, r_{\mathsf{dep}'}, r_{f'}), y)$
12. V: Use above results to verify the sumcheck evaluation of h. If it doesn't verify then output $\perp$
13. V: Output $\top$

Fig. 1. Protocol $\Pi_{\mathsf{VerifyFold}}$

Theorem 1. *Protocol fig. 1 is a* complete *and* sound *interactive argument for the relation* R_{fold}

Proof. Completeness follows directly by inspection. Soundness follows by the same reasoning as that of the GKR protocol [16]. Given a malicious prover that runs the protocol with an $\tilde{O}$ for which it doesn't hold that it's a multilinear extension of a vector $\mathbf{y}$ such that $A\mathbf{x} = \mathbf{y}$ in line with R_{fold} then it's either the case that h and $\tilde{O}$ happen to agree on the randomly verifier-sampled point or there must be a point during one of the invocations of the sumcheck protocol where one of the polynomials sent by the prover happens to agree with the honest polynomial that the prover should have sent. By a union bound over these events we have a soundness error of $O(\frac{\ell+\log\kappa+\log\tau+2\log(d/\tau)}{|\mathbb{F}|})$ by the Schwartz-Zippel lemma where $\mathbb{F}$ is the field over which we run the protocol.

4 Constructing an Inner Product Argument with Succinct Verification for Structured Linear Forms

We now consider how to use the interactive proof from section 3 to construct an inner product argument over $\mathcal{R}_q$ for structured linear relations. Specifically, we consider the following relation

$$R_{\infty,\alpha,L,\delta} = \left\{ P \in \mathcal{R}_q^\kappa, y \in \mathcal{R}_q; \mathbf{x} \in \mathcal{R}^m, \hat{s} \in \mathcal{R}, s \in \mathbb{F}_{q^k} \;\middle|\; \begin{aligned} A\mathbf{x} &= P\hat{s} \\ A &\leftarrow \mathsf{sample}_{\mathsf{poseidon}}(s) \\ L(\mathbf{x}) &= \hat{s}y \\ ||\mathbf{x}||_\infty &\leq \alpha \\ ||\hat{s}||_\infty &\leq \delta \end{aligned} \right\} \tag{34}$$

Where $\hat{s}$ is the *slack* [3] and where L is tensor-structured i. e. it can be written as $[1, v_1] \otimes \cdots \otimes [1, v_\ell]$. Such linear forms can be folded efficiently using essentially the same algorithm as from the previous section. We build a scheme by composing the properly instantiated compressed Σ-protocol with our argument for the correct folding of public parameters in a straight-forward manner. The protocol Π_c is given in Appendix B. As observed in previous work [9], $A \cdot \mathsf{fold}(e_1, \ldots, e_\ell)$ — the culmination of folding the given homomorphism — is not used by the verifier until the final round. We delegate the computation of $A \cdot \mathsf{fold}(e_1, \ldots, e_\ell)$ to the prover.

Theorem 2. *The protocol* Π_{SIPA} *fig. 2 is a* complete *and* knowledge sound *interactive proof for the relation* $R_{\infty,\alpha,L,\delta}$

Proof. Completeness follows directly. For soundness we lean on the analysis of [5] which provides the currently most optimistic analysis that Π_c Appendix B is a knowledge sound interactive proof for relation $R_{\infty,\alpha,L,\delta}$. In it, the authors

Parameters: $A \in \mathcal{R}_q^{\kappa \times m}, y \in \mathcal{R}_q, \alpha \in \mathbb{Z}, \mathcal{C} = \{c \in \mathcal{R} : ||c||_\infty < \beta\} \subset \mathcal{R}$
Prover's input: $\mathbf{x} \in \mathcal{R}_q^m;\ A\mathbf{x} = P \in \mathcal{R}_q^\kappa,\ L(\mathbf{x}) = y$
Claim: $(P, y; \mathbf{x}) \in R_{\infty,\alpha,L,\delta}$

1. PV: Run Π_c Appendix B on input $(P, y; \mathbf{x})$ and public parameters (A, L, α, β) up until verification
2. Let $\mathbf{e} = (e_1, \ldots, e_\ell)$ be the challenges sent by V in the above protocol execution
3. P: Compute and send $a = A \cdot \mathsf{fold}(e_1, \ldots, e_\ell)$ to V
4. PV: Run $\Pi_{\mathsf{VerifyFold}}$ fig. 1 on input $(A, a, s, \mathbf{e})$
5. V: If $\Pi_{\mathsf{VerifyFold}}$ verifies and if Π_c verifies with a then output $\top$ otherwise output $\bot$

Fig. 2. Π_{SIPA} Succinctly Verifiable IPA over $\mathcal{R}_q$

construct an extractor that interacts with the malicious prover to construct a tree of transcripts based on a set of rules from which they can either extract a witness $\mathbf{x} \in \mathcal{R}^m$ and a slackness parameter $\hat{s} \in \mathcal{R}$ such that $\mathbf{x}, \hat{s} \in R_{\infty,\alpha,L,\delta}$ or a witness $\mathbf{x} \in \mathcal{R}^m$ such that $A\mathbf{x} = 0 \mod q$. We now argue that the exact same extractor will succeed on Π_{SIPA} fig. 2 except with negligible probability. This follows from the soundness of $\Pi_{\mathsf{VerifyFold}}$ fig. 1. If $\Pi_{\mathsf{VerifyFold}}$ verifies in the execution of Π_{SIPA} then except with negligible probability the computed folding of A agrees with the correct one and thus the transcript produced during the execution of Π_c in Π_{SIPA} is accepted by the Π_c verifier. By a union bound over all the executions of Π_{SIPA} during extraction — of which there are only polynomially many — all these Π_c transcripts will be accepting except with negligible probability.

4.1 Knowledge Extraction via Tree of Transcripts

The notion of special soundness for 3-step Σ-protocols can be shown to imply knowledge soundness. Special soundness has since been extended to multiple round protocols via witness extraction from a *tree of transcripts*. This is a tree τ where each node corresponds to a prover message and each vertex corresponds to a verifier challenge. A subtree τ' of τ contains the suffixes of a set of individual transcripts that all share the same prefix up to and including the depth of the root of τ' in τ. Recent work expands on this notion [11] [1] [5] by allowing the challenges in these trees of transcripts to depend on more than just the challenges themselves. What they all have in common is that the knowledge extractor can be split into two components; A, which talks to the prover and extracts accepting transcripts and B, which extracts a witness to a given relation R or one of a set of relations $\{R_i\}$ from the extracted tree of transcripts. Thus, for the purposes of this work we need to make sure that if a transcript from our modified multi-round Σ-protocol is accepting then this implies that it contains inside it a transcript that is accepting with respect to the original Σ-protocol. If that's the case then we can simply invoke the knowledge extractor from [5]. Thankfully, this follows from the soundness of $\Pi_{\mathsf{VerifyFold}}$.

4.2 Costs

The protocol inherits the computational and communication complexity of Π_c Appendix B which is polylogarithmic in m [5]. On top of that it also has the cost of an execution of $\Pi_{\mathsf{VerifyFold}}$ fig. 1. The most expensive part of $\Pi_{\mathsf{VerifyFold}}$ for the prover is the sumcheck protocol over h, which due to the number of variables incurs an additional factor $O(d/\tau)$ over $O(\kappa md)$ which is the total number of evaluations defining $\tilde{A}$. This factor could be removed via by employing the techniques from [23]. The polynomial that is summed over has degree at most 3 in each variable and thus the communication complexity of the first round of sumcheck is $O(nmd^2/\tau)$. The Poseidon parameters t, α, R will enforce some concrete costs on the prover and the verifier in the execution of $\Pi_{\mathsf{ParallelPoseidon}}$ fig. 4. α will determine the degree of the polynomials that are summed over. R of course determines the number of invocations of the sumcheck protocol. t contributes quadratically to the complexity of evaluating $\tilde{M}$ which needs to be evaluated once per sumcheck by the verifier. For the sake of this work we consider all these to be constant.

4.3 Fiat-Shamir

All our constructions are public-coin and thus technically compatible with the well-known Fiat-Shamir transformation [15]. Something to note in this regard is a recent result [20] that renders the Fiat-Shamir transformed GKR protocol insecure for a circuit that resembles our computation. While their attack cannot be directly applied to our protocol it should be noted that the Fiat-Shamir transform of our protocol should not be instantiated using Poseidon with the same parameters as used in $\Pi_{\mathsf{VerifyFold}}$ and preferably with an entirely different hash function altogether.

References

1. Aardal, M.A., Aranha, D.F., Boudgoust, K., Kolby, S., Takahashi, A.: Aggregating falcon signatures with LaBRADOR. Cryptology ePrint Archive, Paper 2024/311 (2024), https://eprint.iacr.org/2024/311
2. Ajtai, M.: Generating hard instances of lattice problems (extended abstract). In: Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing. p. 99–108. STOC '96, Association for Computing Machinery, New York, NY, USA (1996). https://doi.org/10.1145/237814.237838, https://doi.org/10.1145/237814.237838
3. Albrecht, M.R., Lai, R.W.F.: Subtractive sets over cyclotomic rings. In: Malkin, T., Peikert, C. (eds.) Advances in Cryptology – CRYPTO 2021. pp. 519–548. Springer International Publishing, Cham (2021)
4. Attema, T., Cramer, R.: Compressed σ-protocol theory and practical application to plug & play secure algorithmics. Cryptology ePrint Archive, Paper 2020/152 (2020), https://eprint.iacr.org/2020/152
5. Attema, T., Klooß, M., Lai, R.W.F., Yatsyna, P.: Adaptive special soundness: Improved knowledge extraction by adaptive useful challenge sampling. Cryptology ePrint Archive, Paper 2024/2038 (2024), https://eprint.iacr.org/2024/2038

6. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: On the indifferentiability of the sponge construction. In: Smart, N. (ed.) Advances in Cryptology – EUROCRYPT 2008. pp. 181–197. Springer Berlin Heidelberg, Berlin, Heidelberg (2008)
7. Bootle, J., Cerulli, A., Chaidos, P., Groth, J., Petit, C.: Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. Cryptology ePrint Archive, Paper 2016/263 (2016), https://eprint.iacr.org/2016/263
8. Bootle, J., Chiesa, A., Sotiraki, K.: Lattice-based succinct arguments for NP with polylogarithmic-time verification. Cryptology ePrint Archive, Paper 2023/930 (2023), https://eprint.iacr.org/2023/930
9. Bowe, S., Grigg, J., Hopwood, D.: Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Paper 2019/1021 (2019), https://eprint.iacr.org/2019/1021
10. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. Cryptology ePrint Archive, Paper 2017/1066 (2017), https://eprint.iacr.org/2017/1066
11. Bünz, B., Fisch, B.: Multilinear schwartz-zippel mod n with applications to succinct arguments. Cryptology ePrint Archive, Paper 2022/458 (2022), https://eprint.iacr.org/2022/458
12. Canetti, R., Jain, A., Scafuro, A.: Practical UC security with a global random oracle. Cryptology ePrint Archive, Paper 2014/908 (2014), https://eprint.iacr.org/2014/908
13. Dutta, M., Ganesh, C., Jawalkar, N.: Succinct verification of compressed sigma protocols in the updatable SRS setting. Cryptology ePrint Archive, Paper 2024/075 (2024), https://eprint.iacr.org/2024/075
14. Eagen, L., Gabizon, A.: Revisiting the IPA-sumcheck connection. Cryptology ePrint Archive, Paper 2025/1325 (2025), https://eprint.iacr.org/2025/1325
15. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) Advances in Cryptology — CRYPTO' 86. pp. 186–194. Springer Berlin Heidelberg, Berlin, Heidelberg (1987)
16. Goldwasser, S., Kalai, Y.T., Rothblum, G.N.: Delegating computation: Interactive proofs for muggles. J. ACM **62**(4) (Sep 2015). https://doi.org/10.1145/2699436, https://doi.org/10.1145/2699436
17. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. Cryptology ePrint Archive, Paper 2019/458 (2019), https://eprint.iacr.org/2019/458
18. Grassi, L., Khovratovich, D., Schofnegger, M.: Poseidon2: A faster version of the poseidon hash function. Cryptology ePrint Archive, Paper 2023/323 (2023), https://eprint.iacr.org/2023/323
19. Huang, M.Y.M., Mao, X., Zhang, J.: Sublinear proofs over polynomial rings. Cryptology ePrint Archive, Paper 2025/199 (2025), https://eprint.iacr.org/2025/199
20. Khovratovich, D., Rothblum, R.D., Soukhanov, L.: How to prove false statements: Practical attacks on fiat-shamir. Cryptology ePrint Archive, Paper 2025/118 (2025), https://eprint.iacr.org/2025/118
21. Lee, J.: Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. Cryptology ePrint Archive, Paper 2020/1274 (2020), https://eprint.iacr.org/2020/1274
22. Lund, C., Fortnow, L., Karloff, H., Nisan, N.: Algebraic methods for interactive proof systems. J. ACM **39**(4), 859–868 (Oct 1992). https://doi.org/10.1145/146585.146605, https://doi.org/10.1145/146585.146605

23. Nguyen, N.K., O'Rourke, G., Zhang, J.: Hachi: Efficient lattice-based multilinear polynomial commitments over extension fields. Cryptology ePrint Archive, Paper 2026/156 (2026), https://eprint.iacr.org/2026/156
24. Nguyen, N.K., Seiler, G.: Greyhound: Fast polynomial commitments from lattices. Cryptology ePrint Archive, Paper 2024/1293 (2024). https://doi.org/10.1007/978-3-031-68403-6˙8, https://eprint.iacr.org/2024/1293
25. Nguyen, W., Setty, S.: Neo and SuperNeo: Post-quantum folding with pay-per-bit costs over small fields. Cryptology ePrint Archive, Paper 2026/242 (2026), https://eprint.iacr.org/2026/242
26. Thaler, J.: Time-optimal interactive proofs for circuit evaluation. Cryptology ePrint Archive, Paper 2013/351 (2013), https://eprint.iacr.org/2013/351
27. Zeilberger, H., Chen, B., Fisch, B.: BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes. Cryptology ePrint Archive, Paper 2023/1705 (2023), https://eprint.iacr.org/2023/1705
28. Zippel, R.: Probabilistic algorithms for sparse polynomials. In: Proceedings of the International Symposiumon on Symbolic and Algebraic Computation. p. 216–226. EUROSAM '79, Springer-Verlag, Berlin, Heidelberg (1979)

A Additional Preliminaries

A.1 Special Soundness, Tree of Transcript Extractors and Norm-Bounded One-way Homomorphisms

Special soundness is a property held by many public-coin proofs of knowledge. For three round protocols for a given relation $\{(x, w)\}$ k-special soundness states that a set of k transcripts of the form $(a, e_1, z_1), \ldots, (a, e_k, z_k)$ there exists a (probabilistic) polynomial time algorithm usually called the *extractor* that can output a witness w for the statement x. This notion is generalized to multiple round protocols via *trees of transcripts*. This is a set of transcripts where vertices correspond to the prover's messages and edges correspond to the verifier's responses. Special soundness has been shown to imply knowledge soundness [7]. Generally, this works by designing an algorithm that can extract such a tree of transcripts from a malicious prover in polynomial time. A recent line of work [11][1][5] explores the gains that can be made if these two processes can interact i. e. if the extraction process of transcripts from a malicious prover is allowed to depend on the progress of the extraction of a witness. This has led to much improved analysis especially in settings where proofs are performed over commitment schemes that only remain binding on norm-bounded values. In these settings knowledge soundness can typically only be enforced in a relaxed sense involving a *stretch* and *slack* [3]. The stretch is an increase on the norm of the extracted witness and the slack is a factor on the element the prover is proving knowledge of a pre-image of. This generally makes the extraction process much harder especially in the setting of classical special soundness where simulated challenges only depend on previous challenges. [3] further show a fundamental impossibility result when using this approach in that challenge spaces that ensure small slack sizes are at most polynomial in size. Recent work circumvents this problem via new techniques and in this work we mainly rely on the results from [5] to argue knowledge soundness of the final construction.

A.2 Poseidon Hash Function

The Poseidon hash function [17][18] is a concrete cryptographic hash function specifically designed to be efficiently arithmetized over prime finite fields namely by constructing it over prime finite fields. It uses a mix of S-box layers and MDS layers to design a pseudorandom permutation. This can either be used in a sponge construction [6] or a compression mode [18]. The hash function is instantiated over four parameters (q, t, R_F, R_P); the size of the field, the number of field elements that the permutation is defined over, the number of full rounds and the number of partial rounds. $R = R_F + R_P$ denotes the full number of rounds. The Poseidon permutation follows the following structure:

$$P(x) = E_{R_F-1} \circ \cdots \circ E_{R_F/2} \circ I_{R_P-1} \circ \cdots \circ I_0 \circ E_{R_F/2-1} \circ \cdots \circ E_0(x) \quad (35)$$

Where E denotes full rounds and I denotes partial rounds. The two types of rounds look as follows:

$$E_i(x) = M \cdot ((x_0 + c_0^{(i)})^d, (x_1 + c_1^{(i)})^d, \ldots, (x_{t-1} + c_{t-1}^{(i)})^d) \tag{36}$$

$$I_i(x) = M \cdot ((x_0 + c_0^{(i)}), (x_1 + c_1^{(i)}), \ldots, (x_{t-1} + c_{t-1}^{(i)})^d) \tag{37}$$

A.3 Random Oracle Model

We recall the notion of a random oracle in line with [12].

Definition 10 (Fixed-size Ideal Random Oracle). *We consider an idealized fixed-size random oracle* RO *to be a function from* $\mathbb{F} \to \mathcal{R}$ *that initially instantiates a memory* M *containing entries of the form* $(\mathbb{F}, \mathcal{R})$*. On input* $x \in \mathbb{F}$ *it checks if* M *contains an entry of the form* (x, y)*. If it does, it returns* y*. Otherwise it samples* $y' \in_R \mathcal{R}$*, stores* (x, y') *in* M *and returns* y'

B Instantiation of Compressed Sigma Protocol

C Instantiation of GKR Protocol for Parallel Evaluations of the Poseidon Hash Function

C.1 Poseidon Sampler

For concreteness, in this work we will consider the concrete instantiation of h using the Poseidon hash function. In particular, we will consider the Poseidon permutation coupled with a truncating compression mode as follows:

$$x \in \mathbb{F}_q^t \to \mathsf{Tr}_n(P(x) + x) \in \mathbb{F}_q^n \tag{38}$$

We use this in line with the NTT sampler:

$$P : \mathbb{F}_q^t \to \mathbb{F}_q^t \tag{39}$$

$$\mathsf{sample}_{\mathsf{poseidon}} : \mathbb{F}_{q^k} \to (\mathbb{F}_{q^{d/\tau}}^{\tau})^{\kappa \times m} \tag{40}$$

$$\mathsf{sample}_{\mathsf{poseidon}} \to \begin{bmatrix} \mathsf{Tr}_{d/\tau}(P(s)) & \mathsf{Tr}_{d/\tau}(P(s^2)) & \ldots & \mathsf{Tr}_{d/\tau}(P(s^{\tau m})) \\ \vdots & \vdots & \vdots & \vdots \\ \mathsf{Tr}_{d/\tau}(P(s^{\tau m(\kappa-1)+1})) & \mathsf{Tr}_{d/\tau}(P(s^{\tau m(\kappa-1)+2})) & \ldots & \mathsf{Tr}_{d/\tau}(P(s^{\tau m \kappa})) \end{bmatrix} \tag{41}$$

Where c is some constant. We heuristically conjecture that sample_P samples sound public parameters for the Ajtai commitment scheme when evaluated on a generator of $\mathbb{F}_{q^k}^*$.

Public Parameters: $A \in \mathcal{R}_q^{\kappa \times m}, y \in \mathcal{R}_q, \alpha, \beta' \in \mathbb{Z}$,
$\mathcal{C} = \{c \in \mathcal{R} : ||c|| < \beta\} \subset \mathcal{R}$
Prover's Input: $\mathbf{x} \in \mathcal{R}_q^m; A\mathbf{x} = P, L(\mathbf{x}) = y$
Verifier's Input: $P \in \mathcal{R}_q^\kappa$

Prover		**Verifier**
$y_a = L_R(\mathbf{x}_L)$		
$y_b = L_L(\mathbf{x}_R)$		
$a = A_R\mathbf{x}_L$		
$b = A_L\mathbf{x}_R$	$\xrightarrow{a,b,y_a,y_b}$	
	$\xleftarrow{e}$	$e \leftarrow \mathcal{C}$
$\mathbf{z} = \mathbf{x}_L + e\mathbf{x}_R$		
if $\|\mathbf{z}\| = 1$:	$\xrightarrow{z}$	$a + eP + e^2 b \stackrel{?}{=}$
		$(eA_L + A_R)\mathbf{z}$
		$y_a + ye + y_b e^2 \stackrel{?}{=}$
		$(eL_L + L_R)(\mathbf{z})$
		$\|\mathbf{z}\|_\infty \stackrel{?}{\leq} \alpha\beta'$
else:		

run Π_c on input $(a + eP + e^2 b; \mathbf{z})$ and public parameters $(eA_L + A_R, eL_L + L_R, \alpha, \beta\beta')$

Fig. 3. Compressed Σ Protocol Π_c

C.2 GKR Poseidon

We wish to reduce a statement about a layer of the computation to a statement about the previous layer. Due to its algebraic structure the Poseidon permutation can be expressed quite easily as a layered circuit. Previous work due the Thaler [26] constructs a very efficient protocol for proving matrix multiplications and adding round constants simply amounts to adding their multilinear extension. Thus, for a full round in the Poseidon permutation we can express the multilinear extension of the input to the next layer $\tilde{V}_i$ via the multilinear extensions of the MDS matrix $\tilde{M}$, the round constants $\tilde{RC}_i$ and the values of the previous layer $\tilde{V_{i+1}}$ as follows

$$\tilde{V}_i(\mathsf{col}, \mathsf{col}') = \sum_{\hat{\mathsf{col}} \in \{0,1\}^{\ell - \log t}, \mathsf{col}'' \in \{0,1\}^{\log t}} \left[\tilde{M}(\mathsf{col}', \mathsf{col}'')(\tilde{V}_{i+1}(\hat{\mathsf{col}}, \mathsf{col}'') + \tilde{\mathsf{RC}}_i(\mathsf{col}''))^\alpha \right] \tilde{\mathsf{eq}}(\mathsf{col}', \mathsf{col}'') \tilde{\mathsf{eq}}(\mathsf{col}, \hat{\mathsf{col}}) \quad (42)$$

Where α is the exponent used in the S-boxes. The partial rounds are a little less straight-forward to express as arithmetic combinations of multilinear polynomials. For a polynomial extension $\tilde{V}$ we can define

$$\tilde{V}'(x, y) = \sum_{b \in \{0,1\}^{\log t}} \left[(1 - \tilde{eq}(1^{\log t}, b))\tilde{V}(x, b) + \tilde{eq}(1^{\log t}, b)\tilde{V}(x, b)^\alpha \right] \tilde{eq}(b, y) \quad (43)$$

This polynomial is such that $\tilde{V}'(x, 1^{\log t}) = \tilde{V}(x, 1^{\log t})^d$ for all boolean x. This corresponds to every t'th evaluation over the boolean hypercube. For all other boolean inputs $\tilde{V}'(x, y) = \tilde{V}(x, y)$. Furthermore it is multilinear. We will incorporate this formulation to prove evaluations of partial rounds by applying the inner part of it to the polynomial $\tilde{V}_{i+1}(\mathsf{col}, \mathsf{col}') + \tilde{RC}_i(\mathsf{col}')$. Call the resulting polynomial $\tilde{R}_i(\mathsf{col}, \mathsf{col}')$. Now we can express the extension of layer i via the extension of layer $i + 1$ as follows:

$$\tilde{V}_i(\mathsf{col}, \mathsf{col}') = \sum_{\hat{\mathsf{col}} \in \{0,1\}^{\ell - \log t}, \mathsf{col}'' \in \{0,1\}^{\log t}} \left[\tilde{M}(\mathsf{col}', \mathsf{col}'')\tilde{R}_i(\mathsf{col}, \mathsf{col}'') \right] \tilde{\mathsf{eq}}(\mathsf{col}', \mathsf{col}'') \tilde{\mathsf{eq}}(\mathsf{col}, \hat{\mathsf{col}}) \quad (44)$$

Compression Following the Poseidon permutation we finally apply a compression mode. One approach presented in the Poseidon2 paper [18] adds the input x to the permutation output $P(x)$ and then truncates the output to the desired number of output field elements

$$x \in \mathbb{F}_q^t \to \mathsf{Tr}_n(P(x) + x) \in \mathbb{F}_q^n \quad (45)$$

This can certainly be combined additively with the final layer as a single sum-check argument over the sufficiently sized boolean hypercube to truncate the output, which won't add to the depth of the circuit.

Parameters: $(A, s, \mathbf{e}, y) \in (\mathbb{F}_q^m, \mathbb{F}_{q^k}, \mathbb{F}_q^l, \mathbb{F}_q)$
Claim: $(A, s, \mathbf{e}, y) \in R_{\mathsf{ParallelPoseidon}}$
Protocol:

1. PV: Define the polynomial:

$$\tilde{A}(x_1, \ldots, x_\ell) = V_0(x_1, \ldots, x_\ell) = V_1(0^{\log n}, x_1, \ldots, x_\ell) + \tilde{S}(0^{\log n}, x_1, \ldots, x_\ell)$$

   where $\tilde{S}$ is the extension of s
2. P: Send $v_1 = V_1(0^{\log n}, r_1, \ldots, r_l)$ to V
3. For i in $[1..R]$ do
   (a) PV: Run sumcheck over $\tilde{V}_i$ to verify P's claimed value of v_i using eq. (42) if i is a full round or eq. (44) if i is partial to represent $\tilde{V}_i$. V can evaluate $\tilde{M}, \tilde{RC}_i$ by itself.
   Let $(e_1^i, \ldots, e_{\ell+\log n}^i)$ be the challenges sent by the verifier during the execution of sumcheck
   (b) P: Send v_{i+1} claimed to equal $V_{i+1}(e_1^i, \ldots, e_{\ell+\log n}^i)$ to V
4. V: Evaluate $\tilde{S}(e_1^{R+1}, \ldots, e_{\ell+\log n}^{R+1})$
5. V: Use this evaluation to verify P's claimed value of v_{R+1} directly
6. V: Evaluate $\tilde{S}(0^{\log n}, \mathbf{e})$
7. V: Check if $V_0(\mathbf{e}) = V_1(0^{\log n}, \mathbf{e}) + \tilde{S}(0^{\log n}, \mathbf{e})$
8. V: If none of the above checks failed, output $\top$, otherwise output $\bot$

Fig. 4. Protocol $\Pi_{\mathsf{ParallelPoseidon}}$

C.3 Putting Things Together: An Interactive Protocol for Evaluating the Multilinear Extension of the Result of Parallel Evaluations of the Poseidon Hash Function

We now describe the full protocol for proving an evaluation of the multilinear extension of the result of m parallel executions of the Poseidon Hash Function. Consider the function

$$\mathsf{ParallelPoseidon} : s \in \mathbb{F}_{q^k} \rightarrow [\mathsf{Tr}_n(P(s) + s), \mathsf{Tr}_n(P(s^2) + s^2), \ldots, \mathsf{Tr}_n(P(s^m) + s^m)] \in \mathbb{F}_q^m \quad (46)$$

We consider the following relation

$$R_{\mathsf{ParallelPoseidon}} = \left\{ (A, s, \mathbf{e}, y) \in (\mathbb{F}_q^m, \mathbb{F}_{q^k}, \mathbb{F}_q^l, \mathbb{F}_q) \;\middle|\; \begin{array}{r} A \leftarrow \mathsf{ParallelPoseidon}(s) \\ \tilde{A}(\mathbf{e}) = y \end{array} \right\} \quad (47)$$

The protocol is given in fig. 4.

Theorem 3. *The protocol fig. 4 is an interactive proof for the relation $R_{\mathsf{ParallelPoseidon}}$ with soundness error $O(\frac{lR}{|\mathbb{F}_q|})$*

Proof. The proof follows the same line of reasoning as that for the GKR protocol. In order for a malicious prover to cheat it has to be the case that at least once

the malicious prover sends a wrong polynomial s in one of the rounds of one of the invocations of the sumcheck protocol such that $s \neq g$ the honest polynomial that the prover should have sent but that this polynomial s happens to agree with g in the following verifier-sampled challenge point. Due to the Schwartz-Zippel lemma this will only happen with probability $O(\frac{l}{|\mathbb{F}_q|})$. By a union bound over all the rounds of the protocol the soundness error is $O(\frac{lR}{|\mathbb{F}_q|})$